

SlashRing — Design profissional da UI (PluginEditor)

Guia para deixar a interface no nível de um plugin comercial, com explicação de cada parte para você implementar à mão e aprender.

Arquivos envolvidos: **Source/PluginEditor.h** e **Source/PluginEditor.cpp** (os dois já existem — você só substitui o conteúdo; não precisa mexer no Projucer).

O que muda em relação ao editor atual

O editor de hoje tem knobs soltos em coordenadas fixas, sem rótulos, sem agrupamento visual, e não expõe vários parâmetros. O redesign traz:

- **LookAndFeel próprio** (SlashRingLookAndFeel): knobs rotativos desenhados por você (arco de valor, corpo do knob, ponteiro) e botões estilo pedal com LED.
- **Componente Knob**: junta um slider + um rótulo, então todo knob já vem com nome (resolve o TD-012).
- **Seções visuais**: painéis com título (INPUT, OVERDRIVE, AMPLIFIER, CABINET, DELAY, REVERB, OUTPUT).
- **Todos os parâmetros expostos**, incluindo os que faltavam: Overdrive Drive/Level, Reverb Mix e o seletor de Input Type (ComboBox).
- **Layout com retângulos** (removeFromTop/removeFromLeft) em vez de coordenadas mágicas — profissional e fácil de ajustar.
- **Paleta Marshall/Slash**: fundo carvão, dourado como cor de destaque, LED vermelho quente.

Conceitos que você vai aprender aqui

- **LookAndFeel**: classe que centraliza o desenho de todos os controles. Você aplica com `setLookAndFeel(&lnf)` no editor, e todos os filhos herdam pela busca na hierarquia. Sempre chame `setLookAndFeel(nullptr)` no destrutor para não deixar ponteiro pendurado.
- **Attachments**: `SliderAttachment`, `ButtonAttachment` e `ComboBoxAttachment` ligam o controle ao parâmetro do APVTS. Você nunca seta valor na mão — o attachment sincroniza os dois lados e cuida da automação.
- **Composição de componentes**: O Knob é um Component que contém um Slider e um Label. Isso evita dezenas de labels soltos e mantém knob+nome sempre juntos.
- **Layout por retângulos**: `getLocalBounds()` e ir "fatiando" com `removeFromTop/removeFromLeft`. Redimensiona bem e é legível.
- **paint() x componentes**: O `paint()` do editor desenha o fundo e os painéis das seções; os filhos (knobs, botões) são desenhados por cima automaticamente.

Mapa visual do layout

SLASHRING									
Slash-voiced guitar suite									
INPUT	OVERDRIVE	AMPLIFIER							
[type v]	(o) OVERDRIVE	GAIN	BASS	MIDDLE	TREBLE	PRESENCE	MASTER		
(INPUT)	DRIVE LEVEL	(o)	(o)	(o)	(o)	(o)	(o)		
CABINET	DELAY	REVERB				OUTPUT			
(o) CAB	(o) DELAY	(o) REVERB							
IR-BASED	TIME FEEDBACK MIX	MIX				(OUTPUT)			

Paleta de cores

- Fundo (gradiente): #231E19 -> #141210
- Painéis: #231F1B, borda #3A352F
- Destaque (dourado): #E0A24A
- LED ativo (vermelho quente): #E0492E
- Texto: #EDE7DD

Arquivo completo: Source/PluginEditor.h

```
#pragma once
#include <JuceHeader.h>
#include "PluginProcessor.h"

//=====
// LOOK & FEEL – desenho central de todos os controles
//=====
class SlashRingLookAndFeel : public juce::LookAndFeel_V4
{
public:
    SlashRingLookAndFeel();

    void drawRotarySlider (juce::Graphics&, int x, int y, int width, int height,
                          float sliderPos, float startAngle, float endAngle,
                          juce::Slider&) override;

    void drawToggleButton (juce::Graphics&, juce::ToggleButton&,
                          bool shouldDrawButtonAsHighlighted,
                          bool shouldDrawButtonAsDown) override;

    const juce::Colour accentColour { 0xffE0A24A }; // dourado
    const juce::Colour trackColour { 0xff3A352F };
    const juce::Colour knobColour { 0xff262220 };
    const juce::Colour textColour { 0xffE7E7DD };
    const juce::Colour ledColour { 0xffE0492E }; // vermelho quente
};

//=====
// KNOB – slider rotativo + rótulo, num unico componente
//=====
class Knob : public juce::Component
{
public:
    explicit Knob (const juce::String& caption);
    void resized() override;

    juce::Slider slider;
    juce::Label label;
};

//=====
// EDITOR
//=====
class SlashRingAudioProcessorEditor final : public juce::AudioProcessorEditor
{
public:
    explicit SlashRingAudioProcessorEditor (SlashRingAudioProcessor&);
    ~SlashRingAudioProcessorEditor() override;

    void paint (juce::Graphics&) override;
    void resized() override;

private:
    SlashRingAudioProcessor& processor;
    SlashRingLookAndFeel lnf;

    // ---- Controles ----
    juce::ComboBox inputTypeBox;
    Knob inputGain { "INPUT" };
    Knob outputGain { "OUTPUT" };

    juce::ToggleButton overdriveButton { "OVERDRIVE" };
    Knob odDrive { "DRIVE" };
    Knob odLevel { "LEVEL" };

    Knob ampGain { "GAIN" };
    Knob bass { "BASS" };
    Knob mid { "MIDDLE" };
    Knob treble { "TREBLE" };
    Knob presence { "PRESENCE" };
    Knob master { "MASTER" };

    juce::ToggleButton cabinetButton { "CABINET" };
    juce::Label cabinetInfo;

    juce::ToggleButton delayButton { "DELAY" };
    Knob delayTime { "TIME" };
    Knob delayFb { "FEEDBACK" };
    Knob delayMix { "MIX" };
};
```

```

juce::ToggleButton reverbButton { "REVERB" };
Knob reverbMix { "MIX" };

// ---- Attachments (ligam controle <-> parametro) ----
using SliderAtt = juce::AudioProcessorValueTreeState::SliderAttachment;
using ButtonAtt = juce::AudioProcessorValueTreeState::ButtonAttachment;
using ComboAtt = juce::AudioProcessorValueTreeState::ComboBoxAttachment;

std::unique_ptr<ComboAtt> inputTypeAtt;
std::unique_ptr<SliderAtt> inputGainAtt, outputGainAtt;
std::unique_ptr<SliderAtt> odDriveAtt, odLevelAtt;
std::unique_ptr<SliderAtt> ampGainAtt, bassAtt, midAtt, trebleAtt, presenceAtt, masterAtt;
std::unique_ptr<SliderAtt> delayTimeAtt, delayFbAtt, delayMixAtt, reverbMixAtt;
std::unique_ptr<ButtonAtt> overdriveAtt, cabinetAtt, delayAtt, reverbAtt;

// ---- Paineis das secoes (desenhados no paint) ----
struct Section { juce::Rectangle<int> bounds; juce::String title; };
std::vector<Section> sections;

JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (SlashRingAudioProcessorEditor)
};

```

Arquivo completo: Source/PluginEditor.cpp

```
#include "PluginEditor.h"

//=====
// LOOK & FEEL
//=====
SlashRingLookAndFeel::SlashRingLookAndFeel()
{
    setColour (juce::Slider::textBoxTextColourId,          textColour);
    setColour (juce::Slider::textBoxOutlineColourId,        juce::Colours::transparentBlack);
    setColour (juce::Slider::textBoxBackgroundColourId,     juce::Colour (0xff17140F));
    setColour (juce::ComboBox::backgroundColourId,         juce::Colour (0xff2A2622));
    setColour (juce::ComboBox::textColourId,               textColour);
    setColour (juce::ComboBox::outlineColourId,            juce::Colour (0xff4A443C));
    setColour (juce::ComboBox::arrowColourId,              accentColour);
    setColour (juce::PopupMenu::backgroundColourId,        juce::Colour (0xff201C18));
    setColour (juce::PopupMenu::textColourId,              textColour);
    setColour (juce::PopupMenu::highlightedBackgroundColourId, accentColour.withAlpha (0.30f));
    setColour (juce::Label::textColourId,                  textColour);
}

void SlashRingLookAndFeel::drawRotarySlider (juce::Graphics& g, int x, int y, int width, int height,
                                              float sliderPos, float startAngle, float endAngle,
                                              juce::Slider&)
{
    auto bounds = juce::Rectangle<float> ((float) x, (float) y, (float) width, (float) height).reduced (6.0f);
    auto radius = juce::jmin (bounds.getWidth(), bounds.getHeight()) * 0.5f;
    auto centre = bounds.getCentre();
    auto toAngle = startAngle + sliderPos * (endAngle - startAngle);
    auto lineW = radius * 0.14f;
    auto arcR = radius - lineW * 0.5f;

    // trilho de fundo (arco completo)
    juce::Path track;
    track.addCentredArc (centre.x, centre.y, arcR, arcR, 0.0f, startAngle, endAngle, true);
    g.setColour (trackColour);
    g.strokePath (track, juce::PathStrokeType (lineW, juce::PathStrokeType::curved, juce::PathStrokeType::rounded));

    // arco de valor (do inicio ate a posicao atual)
    juce::Path value;
    value.addCentredArc (centre.x, centre.y, arcR, arcR, 0.0f, startAngle, toAngle, true);
    g.setColour (accentColour);
    g.strokePath (value, juce::PathStrokeType (lineW, juce::PathStrokeType::curved, juce::PathStrokeType::rounded));

    // corpo do knob
    auto knobR = radius * 0.66f;
    juce::Rectangle<float> knob (0.0f, 0.0f, knobR * 2.0f, knobR * 2.0f);
    knob.setCentre (centre);
    g.setColour (knobColour);
    g.fillEllipse (knob);
    g.setColour (knobColour.brighter (0.18f));
    g.drawEllipse (knob, 1.0f);

    // ponteiro
    juce::Path pointer;
    const float thickness = 2.6f;
    pointer.addRoundedRectangle (-thickness * 0.5f, -knobR + 3.0f, thickness, knobR * 0.6f, 1.0f);
    pointer.applyTransform (juce::AffineTransform::rotation (toAngle).translated (centre.x, centre.y));
    g.setColour (accentColour.brighter (0.25f));
    g.fillPath (pointer);
}

void SlashRingLookAndFeel::drawToggleButton (juce::Graphics& g, juce::ToggleButton& b,
                                              bool shouldDrawButtonAsHighlighted, bool)
{
    auto bounds = b.getLocalBounds().toFloat().reduced (1.0f);
    const bool on = b.getToggleState();

    g.setColour (on ? accentColour.withAlpha (0.16f) : juce::Colour (0xff2A2622));
    g.fillRoundedRectangle (bounds, 5.0f);
    g.setColour (on ? accentColour : juce::Colour (0xff4A443C));
    g.drawRoundedRectangle (bounds, 5.0f, shouldDrawButtonAsHighlighted ? 1.6f : 1.0f);

    // LED
    auto led = juce::Rectangle<float> (0.0f, 0.0f, 9.0f, 9.0f)
        .withCentre ({ bounds.getX() + 15.0f, bounds.getCentreY() });
    if (on)
    {
        g.setColour (ledColour.withAlpha (0.35f));
        g.fillEllipse (led.expanded (3.5f));
    }
}
```

```

g.setColour (on ? ledColour : juce::Colour (0xff3A352F));
g.fillEllipse (led);

// texto
juce::Font f (13.0f); f.setBold (true);
g.setFont (f);
g.setColour (on ? textColour : textColour.withAlpha (0.55f));
g.drawText (b.getButtonText(), bounds.withTrimmedLeft (28.0f), juce::Justification::centredLeft, true);
}

//=====
// KNOB
//=====
Knob::Knob (const juce::String& caption)
{
    slider.setSliderStyle (juce::Slider::RotaryHorizontalVerticalDrag);
    slider.setTextBoxStyle (juce::Slider::TextBoxBelow, false, 62, 16);
    addAndMakeVisible (slider);

    label.setText (caption, juce::dontSendNotification);
    label.setJustificationType (juce::Justification::centred);
    label.setInterceptsMouseClicks (false, false);
    juce::Font lf (11.0f); lf.setBold (true);
    label.setFont (lf);
    addAndMakeVisible (label);
}

void Knob::resized()
{
    auto r = getLocalBounds();
    label.setBounds (r.removeFromTop (16));
    slider.setBounds (r);
}

//=====
// EDITOR - construtor
//=====
SlashRingAudioProcessorEditor::SlashRingAudioProcessorEditor (SlashRingAudioProcessor& p)
    : AudioProcessorEditor (&p), processor (p)
{
    setLookAndFeel (&lnf);

    // ComboBox de tipo de captador (IDs 1..5 = indices 0..4 do parametro)
    inputTypeBox.addItem ("Single Coil", 1);
    inputTypeBox.addItem ("P90", 2);
    inputTypeBox.addItem ("Humbucker Vintage", 3);
    inputTypeBox.addItem ("Humbucker Modern", 4);
    inputTypeBox.addItem ("Active", 5);
    addAndMakeVisible (inputTypeBox);

    cabinetInfo.setText ("IR-BASED", juce::dontSendNotification);
    cabinetInfo.setJustificationType (juce::Justification::centred);
    cabinetInfo.setColour (juce::Label::textColourId, juce::Colour (0xff9A9188));
    addAndMakeVisible (cabinetInfo);

    for (auto* k : { &inputGain, &outputGain, &odDrive, &odLevel, &ampGain, &bass,
                    &mid, &treble, &presence, &master, &delayTime, &delayFb, &delayMix, &reverbMix })
        addAndMakeVisible (*k);

    for (auto* btn : { &overdriveButton, &cabinetButton, &delayButton, &reverbButton })
        addAndMakeVisible (*btn);

    // Attachments
    auto& s = processor.getAPVTS();
    inputTypeAtt = std::make_unique<ComboAtt> (s, ParameterID::inputType, inputTypeBox);
    inputGainAtt = std::make_unique<SliderAtt> (s, ParameterID::inputGain, inputGain.slider);
    outputGainAtt = std::make_unique<SliderAtt> (s, ParameterID::outputGain, outputGain.slider);
    odDriveAtt = std::make_unique<SliderAtt> (s, ParameterID::overdriveDrive, odDrive.slider);
    odLevelAtt = std::make_unique<SliderAtt> (s, ParameterID::overdriveLevel, odLevel.slider);
    ampGainAtt = std::make_unique<SliderAtt> (s, ParameterID::ampGain, ampGain.slider);
    bassAtt = std::make_unique<SliderAtt> (s, ParameterID::bass, bass.slider);
    midAtt = std::make_unique<SliderAtt> (s, ParameterID::middle, mid.slider);
    trebleAtt = std::make_unique<SliderAtt> (s, ParameterID::treble, treble.slider);
    presenceAtt = std::make_unique<SliderAtt> (s, ParameterID::presence, presence.slider);
    masterAtt = std::make_unique<SliderAtt> (s, ParameterID::master, master.slider);
    delayTimeAtt = std::make_unique<SliderAtt> (s, ParameterID::delayTime, delayTime.slider);
    delayFbAtt = std::make_unique<SliderAtt> (s, ParameterID::delayFeedback, delayFb.slider);
    delayMixAtt = std::make_unique<SliderAtt> (s, ParameterID::delayMix, delayMix.slider);
    reverbMixAtt = std::make_unique<SliderAtt> (s, ParameterID::reverbMix, reverbMix.slider);
    overdriveAtt = std::make_unique<ButtonAtt> (s, ParameterID::overdriveOn, overdriveButton);
    cabinetAtt = std::make_unique<ButtonAtt> (s, ParameterID::cabinetOn, cabinetButton);
    delayAtt = std::make_unique<ButtonAtt> (s, ParameterID::delayOn, delayButton);
    reverbAtt = std::make_unique<ButtonAtt> (s, ParameterID::reverbOn, reverbButton);

    setSize (1080, 600);
}

```

```

}

SlashRingAudioProcessorEditor::~SlashRingAudioProcessorEditor()
{
    setLookAndFeel (nullptr);
}

//=====
// PAINT – fundo, cabeçalho e painéis das secões
//=====
void SlashRingAudioProcessorEditor::paint (juce::Graphics& g)
{
    juce::ColourGradient grad (juce::Colour (0xff231E19), 0.0f, 0.0f,
                                juce::Colour (0xff141210), 0.0f, (float) getHeight(), false);
    g.setGradientFill (grad);
    g.fillAll();

    // cabeçalho
    auto header = getLocalBounds().removeFromTop (74);
    g.setColour (juce::Colour (0xffE0A24A));
    juce::Font title (34.0f); title.setBold (true);
    g.setFont (title);
    g.drawText ("SLASHRING", header.reduced (22, 0).withTrimmedBottom (12),
                juce::Justification::centredLeft, false);

    g.setColour (juce::Colour (0xffE0E7DD).withAlpha (0.55f));
    g.setFont (juce::Font (12.5f));
    g.drawText ("Slash-voiced guitar suite", header.reduced (24, 0).removeFromBottom (20),
                juce::Justification::centredLeft, false);

    g.setColour (juce::Colour (0xff4A443C));
    g.fillRect (14, 72, getWidth() - 28, 1);

    // painéis das secões
    for (auto& sec : sections)
    {
        auto b = sec.bounds.toFloat();
        g.setColour (juce::Colour (0xff231F1B));
        g.fillRoundedRectangle (b, 8.0f);
        g.setColour (juce::Colour (0xff3A352F));
        g.drawRoundedRectangle (b, 8.0f, 1.0f);

        auto head = sec.bounds.removeFromTop (24); // copia local via valor
        g.setColour (juce::Colour (0xffE0A24A));
        juce::Font hf (12.0f); hf.setBold (true);
        g.setFont (hf);
        g.drawText (sec.title, head.reduced (12, 0), juce::Justification::centredLeft, false);
    }
}

//=====
// RESIZED – layout por retângulos
//=====
void SlashRingAudioProcessorEditor::resized()
{
    sections.clear();

    auto area = getLocalBounds();
    area.removeFromTop (74); // cabeçalho (desenhado no paint)
    area.reduce (14, 10); // margem externa
    const int gap = 12;

    // ----- LINHA A -----
    auto rowA = area.removeFromTop (250);
    area.removeFromTop (gap);
    auto rowB = area; // resto

    auto inputArea = rowA.removeFromLeft (180); rowA.removeFromLeft (gap);
    auto odArea = rowA.removeFromLeft (220); rowA.removeFromLeft (gap);
    auto ampArea = rowA; // resto (o mais largo)

    // ----- LINHA B -----
    auto cabArea = rowB.removeFromLeft (180); rowB.removeFromLeft (gap);
    auto dlyArea = rowB.removeFromLeft (360); rowB.removeFromLeft (gap);
    auto revArea = rowB.removeFromLeft (200); rowB.removeFromLeft (gap);
    auto outArea = rowB; // resto

    // regista os painéis (para o paint desenhar)
    sections.clear();
    sections.push_back ({ inputArea, "INPUT" });
    sections.push_back ({ odArea, "OVERDRIVE" });
    sections.push_back ({ ampArea, "AMPLIFIER" });
    sections.push_back ({ cabArea, "CABINET" });
    sections.push_back ({ dlyArea, "DELAY" });
    sections.push_back ({ revArea, "REVERB" });
}

```

```

sections.push_back ({ outArea, "OUTPUT" });

const int headH = 24;
auto inner = [&] (juce::Rectangle<int> s) { s.removeFromTop (headH); return s.reduced (10); };

// INPUT: combo + knob
{
    auto r = inner (inputArea);
    inputTypeBox.setBounds (r.removeFromTop (26));
    r.removeFromTop (8);
    inputGain.setBounds (r.withSizeKeepingCentre (juce::jmin (r.getWidth(), 96),
                                                         juce::jmin (r.getHeight(), 104)));
}

// OVERDRIVE: toggle + 2 knobs
{
    auto r = inner (odArea);
    overdriveButton.setBounds (r.removeFromTop (28));
    r.removeFromTop (6);
    int kw = r.getWidth() / 2;
    odDrive.setBounds (r.removeFromLeft (kw).reduced (2));
    odLevel.setBounds (r.reduced (2));
}

// AMPLIFIER: 6 knobs
{
    auto r = inner (ampArea);
    juce::Component* ks[] = { &ampGain, &bass, &mid, &treble, &presence, &master };
    const int n = 6;
    int kw = r.getWidth() / n;
    for (int i = 0; i < n; ++i)
        ks[i]->setBounds (r.removeFromLeft (kw).reduced (3));
}

// CABINET: toggle + info
{
    auto r = inner (cabArea);
    cabinetButton.setBounds (r.removeFromTop (28));
    r.removeFromTop (8);
    cabinetInfo.setBounds (r.removeFromTop (24));
}

// DELAY: toggle + 3 knobs
{
    auto r = inner (dlyArea);
    delayButton.setBounds (r.removeFromTop (28));
    r.removeFromTop (6);
    int kw = r.getWidth() / 3;
    delayTime.setBounds (r.removeFromLeft (kw).reduced (2));
    delayFb.setBounds (r.removeFromLeft (kw).reduced (2));
    delayMix.setBounds (r.reduced (2));
}

// REVERB: toggle + 1 knob
{
    auto r = inner (revArea);
    reverbButton.setBounds (r.removeFromTop (28));
    r.removeFromTop (6);
    reverbMix.setBounds (r.withSizeKeepingCentre (juce::jmin (r.getWidth(), 96),
                                                         juce::jmin (r.getHeight(), 104)));
}

// OUTPUT: 1 knob
{
    auto r = inner (outArea);
    outputGain.setBounds (r.withSizeKeepingCentre (juce::jmin (r.getWidth(), 96),
                                                         juce::jmin (r.getHeight(), 104)));
}

repaint();
}

```

Explicacao parte por parte

SlashRingLookAndFeel

É a classe que dá a "cara" ao plugin. Ao herdar de `LookAndFeel_V4` e sobrescrever `drawRotarySlider` e `drawToggleButton`, todo slider rotativo e todo toggle do plugin passam a ser desenhados por você.

- `drawRotarySlider` recebe `sliderPos` (0..1 = posição normalizada do knob) e os ângulos de início/fim. A conta `toAngle = startAngle + sliderPos * (endAngle - startAngle)` converte o valor no ângulo do ponteiro e do arco. Desenhamos: trilho de fundo, arco de valor (dourado), corpo do knob e ponteiro.
- `drawToggleButton` desenha um "pedal" com LED: quando ligado, LED vermelho aceso + moldura dourada; desligado, apagado e texto mais fraco.
- As cores ficam como membros, então trocar o tema é trivial (mude os hex no header).

Knob (composicao)

Em vez de espalhar labels pela tela, cada knob é um `Component` que já contém o `Slider` e o `Label`. No `resized()` do Knob, o rótulo fica em cima (16 px) e o slider ocupa o resto. Assim, onde quer que você posicione um knob, o nome vai junto.

Attachments

Cada controle é ligado ao parâmetro do APVTS por um attachment guardado em `unique_ptr`. Isso faz a sincronização automática (UI <-> DSP <-> automação do host). Repare que os attachments apontam para `knob.slider` (o slider dentro do Knob). Para o `ComboBox`, os IDs dos itens (1..5) correspondem aos índices 0..4 do parâmetro de escolha.

paint()

Desenha o fundo em gradiente, o cabeçalho (título dourado + subtítulo) e, para cada seção registrada, um painel arredondado com título. Como isso roda no `paint()` do editor, os knobs e botões (que são filhos) aparecem por cima.

resized() — layout por retangulos

A técnica profissional: pegue `getLocalBounds()` e vá "fatiando". `removeFromTop(74)` tira o cabeçalho; `reduce(14, 10)` cria margem; `removeFromLeft(180)` corta a coluna da seção. Dentro de cada seção, o `lambda inner()` remove a faixa do título e aplica padding. Ajustar o layout é só mudar esses números — nada de coordenadas mágicas espalhadas.

Como implementar a maõ, em passos (para aprender)

Faça em etapas, compilando a cada uma — você entende cada peça e nunca acumula erro:

1. **Só o LookAndFeel do knob.** Substitua o header e o cpp, mas comece com o layout antigo se quiser. Aplique `setLookAndFeel(&lnf)` e veja os knobs mudarem de aparência. Compile.
2. **O componente Knob.** Troque um slider solto por um knob e confirme que o rótulo aparece e o attachment funciona (aponte para `.slider`). Compile.
3. **Converta os demais controles** para Knob e adicione os que faltavam (Overdrive Drive/Level, Reverb Mix, Input Type). Compile.
4. **paint() das seções + resized() por retângulos.** Coloque os painéis e o layout em grade. Compile e ajuste os números até ficar do seu gosto.
5. **drawToggleButton** (pedal com LED). Compile e teste ligando/desligando.

Ajustes rapidos que voce vai querer fazer

- **Cores/tema:** mude os hex nos membros do `SlashRingLookAndFeel`.
- **Tamanho da janela:** `setSize(1080, 600)` no fim do construtor.
- **Tamanho dos knobs:** OS 96 / 104 NOS `withSizeKeepingCentre` e O `reduced()` dentro das seções.
- **Rótulos das seções:** OS textos em `sections.push_back({ ..., "TITULO" })`.

Observacoes

- Não precisa mexer no Projucer: `PluginEditor.h` e `.cpp` já existem no projeto; você só troca o conteúdo.
- Se quiser um visual ainda mais avançado depois (imagens de fundo, knobs com textura), o caminho é carregar imagens via `BinaryData` e desenhá-las no `drawRotarySlider/paint`. A estrutura aqui já está pronta para isso.
- Compatível com a versão de JUCE do projeto (APIs clássicas). Se o seu JUCE for o 8 e reclamar de `juce::Font(float)`, troque por `juce::Font(juce::FontOptions().withHeight(13.0f))`.